In [34]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv("loan_approval_dataset.csv")
```

In [35]:
```
data.head(5)
```

Out[35]:

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_
0	1	2	Graduate	No	9600000	29900000	12	778	24
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	27
2	3	3	Graduate	No	9100000	29700000	20	506	71
3	4	3	Graduate	No	8200000	30700000	8	467	182
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	124

In [36]:
```
data
```

Out[36]:

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_ass
0	1	2	Graduate	No	9600000	29900000	12	778	
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	
2	3	3	Graduate	No	9100000	29700000	20	506	
3	4	3	Graduate	No	8200000	30700000	8	467	
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	
...	...	...	...	...	...	...	...	...	
4264	4265	5	Graduate	Yes	1000000	2300000	12	317	
4265	4266	0	Not Graduate	Yes	3300000	11300000	20	559	
4266	4267	2	Not Graduate	No	6500000	23900000	18	457	
4267	4268	1	Not Graduate	No	4100000	12800000	8	780	
4268	4269	1	Graduate	No	9200000	29700000	10	607	

4269 rows × 13 columns

In [37]:
```
obj = (data.dtypes == 'object')
print("Categorical variables:",len(list(obj[obj].index)))
```

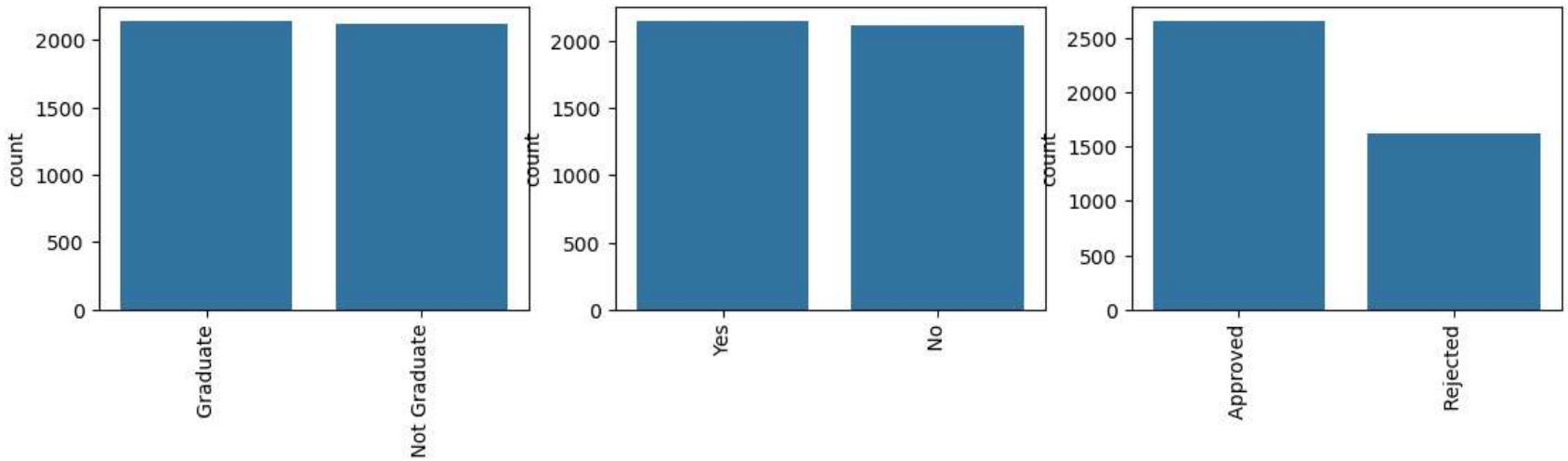
```
Categorical variables: 3
```

In [38]:
```
# Dropping loan_id column
data.drop(['loan_id'],axis=1,inplace=True)
```

In [39]:
```
obj = (data.dtypes == 'object')
object_cols = list(obj[obj].index)
plt.figure(figsize=(18,36))
```

```
index = 1

for col in object_cols:
  y = data[col].value_counts()
  plt.subplot(11,4,index)
  plt.xticks(rotation=90)
  sns.barplot(x=list(y.index), y=y)
  index +=1
```



In [40]:
```
# Import label encoder
from sklearn import preprocessing

# label_encoder object knows how
# to understand word labels.
label_encoder = preprocessing.LabelEncoder()
obj = (data.dtypes == 'object')
for col in list(obj[obj].index):
  data[col] = label_encoder.fit_transform(data[col])
```

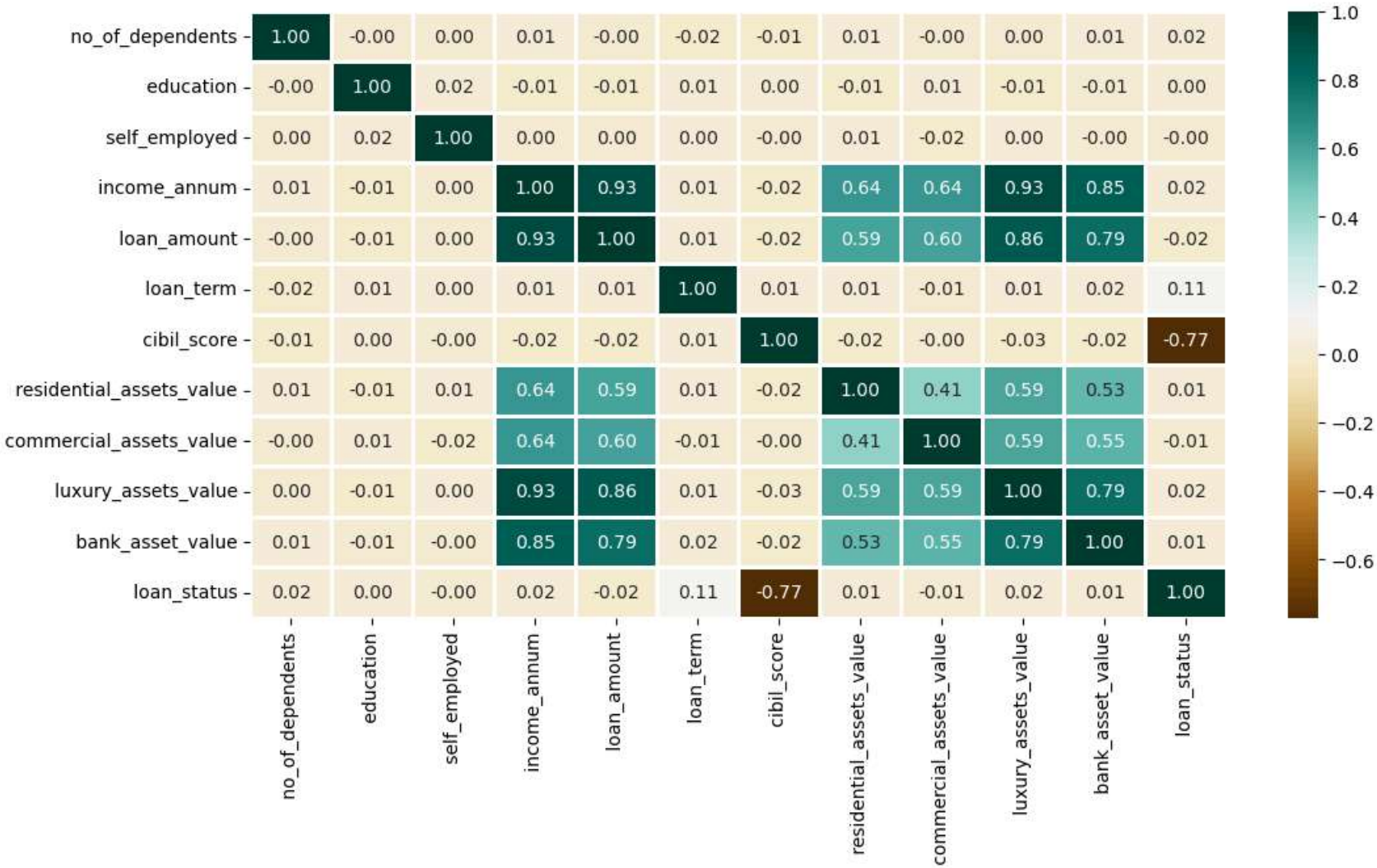
In [41]:
```
# To find the number of columns with
# datatype==object
obj = (data.dtypes == 'object')
print("Categorical variables:",len(list(obj[obj].index)))
```

```
Categorical variables: 0
```

In [42]:
```
plt.figure(figsize=(12,6))

sns.heatmap(data.corr(),cmap='BrBG',fmt='.2f',
            linewidths=2,annot=True)
```

Out[42]: <Axes: >



In [43]:
```
for col in data.columns:
  data[col] = data[col].fillna(data[col].mean())
```

```
data.isna().sum()
```

Out[43]:
```
no_of_dependents            0
education                   0
self_employed               0
income_annum                0
loan_amount                 0
loan_term                   0
cibil_score                 0
residential_assets_value    0
commercial_assets_value     0
luxury_assets_value         0
bank_asset_value            0
loan_status                 0
dtype: int64
```

In [44]:
```
from sklearn.model_selection import train_test_split

X = data.drop([' loan_status'],axis=1)
Y = data[' loan_status']
X.shape,Y.shape

X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
                                                    test_size=0.4,
                                                    random_state=1)
X_train.shape, X_test.shape, Y_train.shape, Y_test.shape
```

Out[44]:
```
((2561, 11), (1708, 11), (2561,), (1708,))
```

In [45]:
```
data[' education'] = data[' education'].map({
    'Graduate': 1,
    'Not Graduate': 0
})
```

In [46]:
```
data[' self_employed'] = data[' self_employed'].map({'Yes': 1, 'No': 0})
```

In [47]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
```

```
from sklearn import metrics

knn = KNeighborsClassifier(n_neighbors=3)
rfc = RandomForestClassifier(n_estimators = 7,
                             criterion = 'entropy',
                             random_state =7)
svc = SVC()
lc = LogisticRegression()

# making predictions on the training set
for clf in (rfc, knn, svc,lc):
    clf.fit(X_train, Y_train)
    Y_pred = clf.predict(X_train)
    print("Accuracy score of ",
          clf.__class__.__name__,
          "=",100*metrics.accuracy_score(Y_train,
                                         Y_pred))
```

```
Accuracy score of  RandomForestClassifier = 99.72666926981648
Accuracy score of  KNeighborsClassifier = 77.31354939476766
Accuracy score of  SVC = 61.772745021475984
Accuracy score of  LogisticRegression = 71.76884029675907
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:465: ConvergenceWarning: lbfgs failed to
converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

In [48]:

```
# making predictions on the testing set
for clf in (rfc, knn, svc,lc):
    clf.fit(X_train, Y_train)
    Y_pred = clf.predict(X_test)
    print("Accuracy score of ",
          clf.__class__.__name__,"=",
          100*metrics.accuracy_score(Y_test,
                                     Y_pred))
```

```
Accuracy score of  RandomForestClassifier = 97.71662763466043
Accuracy score of  KNeighborsClassifier = 56.030444964871194
Accuracy score of  SVC = 62.880562060889936
Accuracy score of  LogisticRegression = 75.0
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:465: ConvergenceWarning: lbfgs failed to
converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

In [ ]: